

SmartCode

An AI-powered adaptive programming tutor built for personalized problem-solving and real learning growth.

SOFTWARE ENGINEERING GRADUATION PROJECT



Made with GAMMA



The Problem with "One Size Fits All"

Meet Two Students

Alex has prior coding experience — he flies through exercises and builds confidence fast. Jordan is just starting out — she gets stuck, reruns the same errors, and eventually disengages.

Both are using the **same platform**, following the **same path**.

No Personalization

Platforms give everyone the same problem set regardless of skill gaps or learning pace.

No Adaptive Feedback

Struggling learners receive no targeted hints or guidance — just a wall of failed attempts.

No Progress Insight

Neither student knows where they truly stand or what to focus on next.

Introducing SmartCode

SmartCode doesn't just give you problems to solve — it builds your ability to *think* through them.



Skill Diagnostics

Evaluates each learner's starting point and maps a personalized curriculum from day one.



Adaptive Progression

Dynamically adjusts problem difficulty and topic order based on ongoing performance.



Intelligent Hints

Delivers contextual, Socratic hints when users are stuck — guiding thinking, not giving answers.



AI Tutoring Sessions

Live conversational tutoring powered by LLMs to walk through concepts interactively.



Performance Analytics

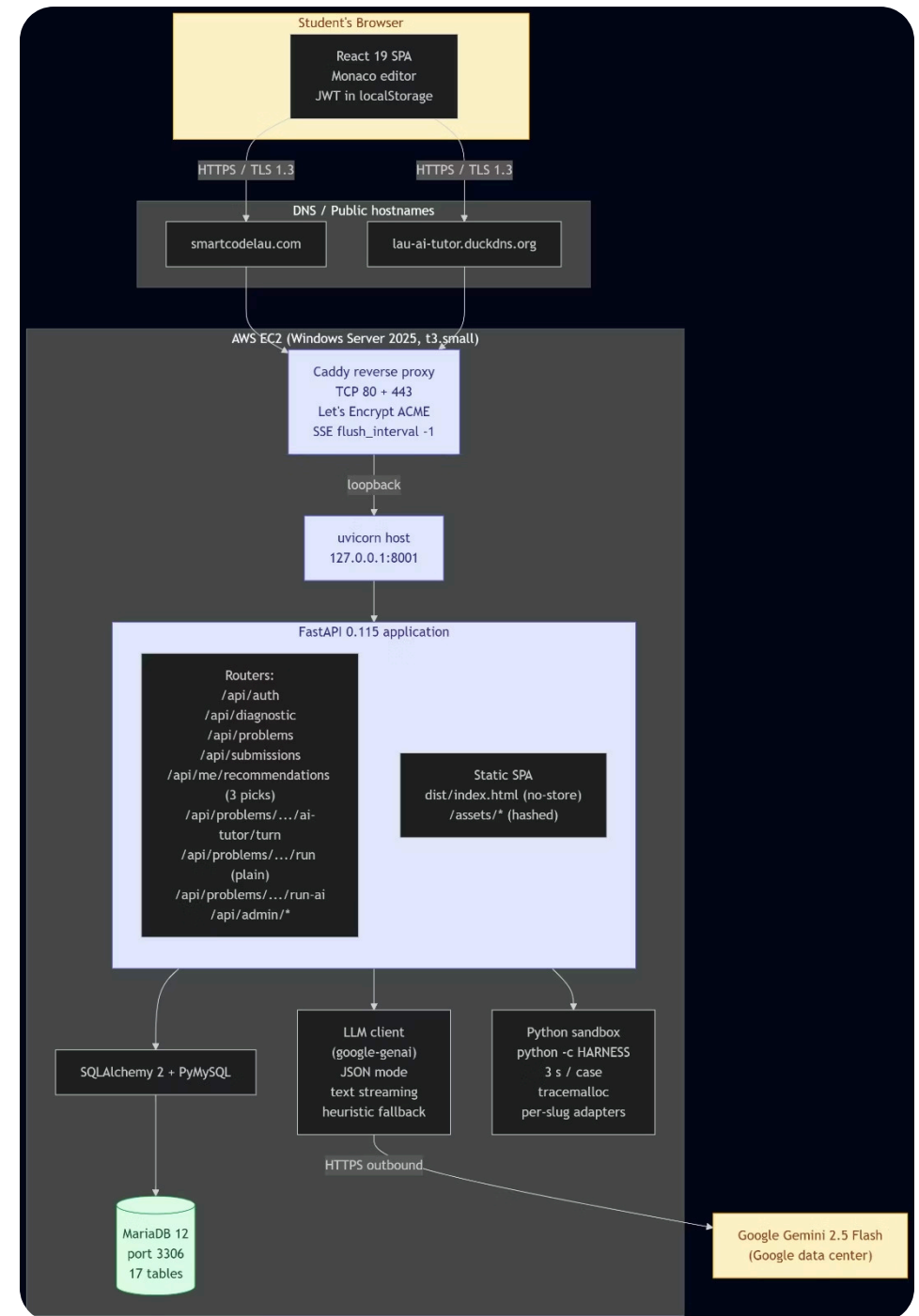
Tracks skill growth over time, surfacing detailed feedback and progress insights for every learner.

System Architecture

How It All Connects

The **frontend** communicates exclusively through a RESTful **backend API**, which orchestrates all core services.

- **PostgreSQL** stores all user data, submissions, and skill history
- **LLM services** power hints, tutoring, and intelligent code evaluation
- A **sandboxed executor** safely runs and tests user-submitted code
- The **recommendation engine** continuously refines each learner's path
- **Analytics** aggregates metrics to surface meaningful progress data



Engineering the Right Way

Scrum + XP Methodology

We adopted a hybrid of **Scrum** for structure and **Extreme Programming (XP)** for engineering discipline — a combination that matched the evolving, feedback-driven nature of SmartCode perfectly.

- Iterative sprints let us validate features early, catch issues fast, and continuously refine the AI behaviors that are hardest to get right the first time.

01

Sprint Planning

Defined clear goals and deliverables every sprint cycle to stay focused and aligned.

02

Pair Collaboration

Critical modules were developed and reviewed together to reduce bugs and share knowledge.

03

Continuous Feedback

Regular retrospectives and peer demos drove rapid iteration on features.

04

Test-Driven Refinement

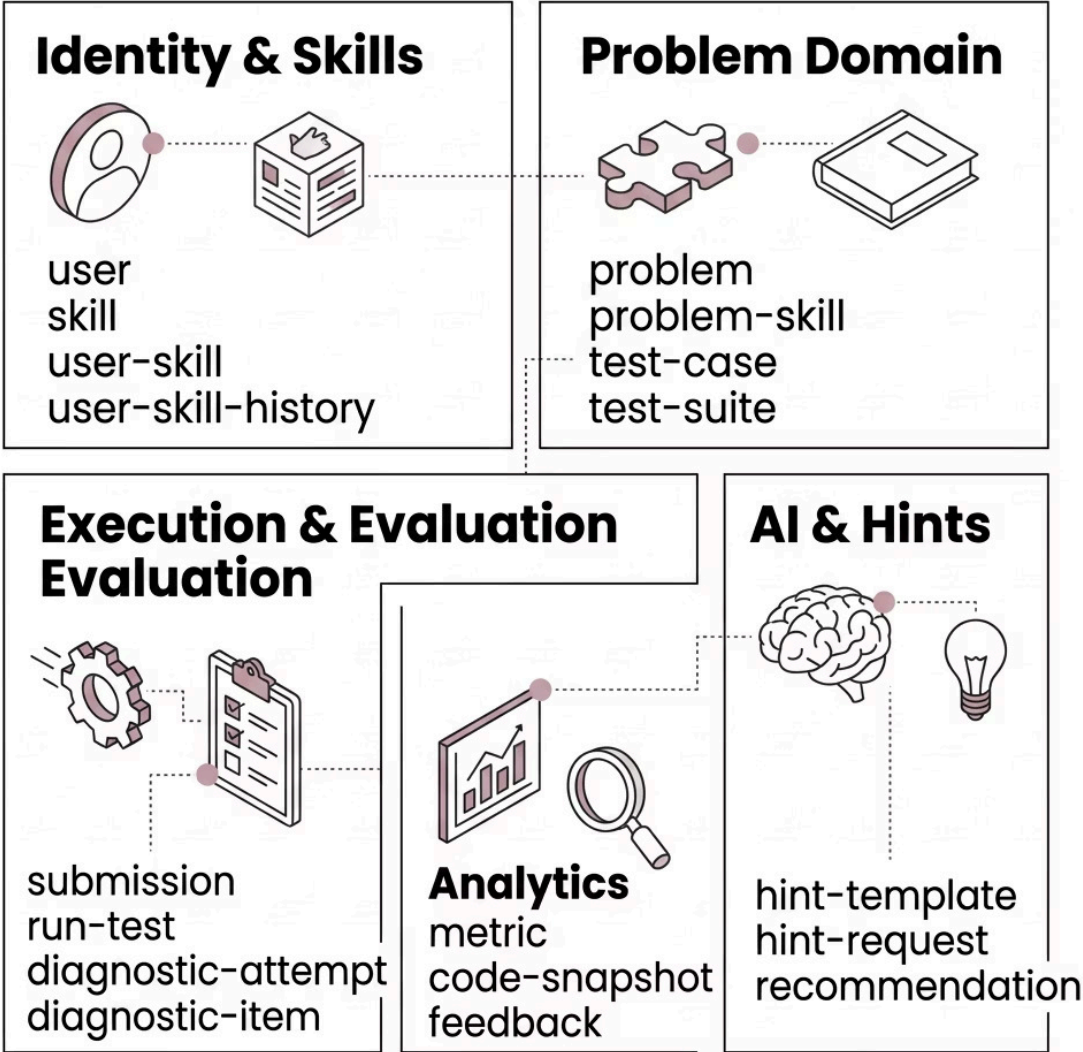
Testing was built into every cycle — not an afterthought — ensuring system reliability.



Database Design

Designed for Personalization at Scale

The database is the backbone of SmartCode's intelligence. Every interaction is captured, stored, and used to drive personalized decisions.



Skill Tracking

users, skills, user-skill, user-skills-history — maps what each learner knows and how it evolves.



Problem & Evaluation

problems, problem-skills, test-cases, test-suites, submissions, run-tests — links problems to skills and evaluates correctness.



AI Interactions

hint-templates, hint-requests, diagnostic-attempts, diagnostic-items — records every AI-guided interaction for learning context.



Analytics & Feedback

metrics, code-snapshots, feedback, recommendations — powers performance insights and adaptive recommendations.

Requirements: What We Promised

Functional Requirements (FRs)

- **User Authentication** — Login and registration with secure credentials
- **Initial Assessment** — Skill-level evaluation on first login
- **Personalized Learning Path** — Adaptive problem recommendations based on skill level
- **Problem Solving** — View, solve, and submit code solutions
- **Code Evaluation** — Automated testing with verdicts (Accepted, Wrong Answer, Error)
- **Adaptive Hints** — Context-aware guidance during problem solving
- **Progress Tracking** — View solved problems, performance metrics, and skill development
- **Smart Recommendations** — Suggest problems based on performance; avoid repetition except for spaced review
- **Submission History** — Maintain and display past attempts

Non-Functional Requirements (NFRs)

Endpoint	Median Latency	Notes
POST /api/auth/login	< 100 ms	Encrypt verify dominates
POST /api/diagnostic	0–2 s	Single Gemini call grades all five items
POST /api/submissions (accepted, easy)	3–7 s	Sandbox under 1 s; Gemini grading 2–6 s
POST /api/submissions (accepted, hard)	5–10 s	More cases; same Gemini latency
GET /api/problems/{id}/hint	1–3 s	Small Gemini call within in-progress code
POST /api/tutor/turn	2–4 s	Stateless; full chat replayed each call
GET /api/me/skills	< 80 ms	Pure MariaDB SELECT, no LLM

Availability: 99% uptime (excluding maintenance) | **Concurrency:** 100+ concurrent users | **Page Load:** < 1 second for 95% of requests

Testing: Finding the Gaps

Alpha Testing — Internal Issues Found

- Weak passwords accepted without validation
- Lucky-coincidence solutions incorrectly marked correct
- LLM evaluator hallucinated correctness on edge cases
- Partial-pass submissions labeled as fully accepted
- Editing re-opened already-solved problems unexpectedly
- Submissions tab displayed incorrect state in some flows

Beta Testing — Real User Feedback

- Diagnostic gate could be bypassed with direct URL access
- Recommendation engine showed limited variety in early stages
- Python syntax errors confused beginners without enough guidance
- AI walkthrough delivered lectures instead of guiding discovery
- FortiGuard firewall blocked the duckdns deployment domain
- Problem catalogue felt limited for advanced learners

✓ Every issue discovered was resolved — each one making SmartCode measurably more robust and learner-friendly.



Challenges We Faced Testing Revealed Critical Issues

- Weak password validation gaps
- Lucky-coincidence solutions marked as correct
- LLM evaluator hallucinations on edge cases
- Partial-pass submissions incorrectly labeled as accepted
- Problem state reset when editing re-opened solved problems
- Submission tab state inconsistencies across user flows

If We Started Over



Windows Deployment Pain

NSSM and PowerShell quirks, file-locking conflicts, and encoding bugs made Windows Server far more painful than expected.



Deploy Earlier

Going live sooner would have unlocked more beta-testing cycles and surfaced real-user issues before final polish.



Secure the Domain Early

Using a free duckdns domain caused firewall blocking for some users.



Linux + Docker from Day One

Docker containers on Linux would have simplified sandbox isolation, dependency management, and the entire deployment pipeline significantly.

Key Takeaways

Real Problem Solved

SmartCode addresses a genuine gap — learners need adaptive, intelligent support, not static problem lists.

AI Done Thoughtfully

LLMs were integrated with guardrails — evaluation, hinting, and tutoring are guided by design, not left to chance.

Built to Learn from Users

The system continuously tracks skill data and feedback, meaning SmartCode improves the more it is used.

Engineering Discipline Matters

Scrum, XP, thorough testing, and honest retrospectives made us a stronger team and a better product.

"The goal was never just to build a platform that gives problems — it was to build one that actually makes you better at solving them."



Thank You

We're proud of what SmartCode became — a system that genuinely adapts to learners, grows with them, and teaches them to think.

QUESTIONS WELCOME